Task W09

We experiment with our first real single-server, multiple-client system with sockets in this task. It helps to read the sockets chapter before this assignment, but you do not need a full understanding to complete it.

Part A – Server and Client

In your Linux environment (either the Xubuntu VM or your own Linux box), complete the following. You can use any text editor you wish. I used vim in an alacritty terminal.

1) Create a directory named taskw09 (you will have code, screenshots, and other files in this directory)

2) Create CounterServer.java as shown in the image 01CounterServer.png

3) Create CounterClient.java as shown in the image 02CounterClient.png

4) Compile the two java files at the command line

5) In one terminal, run the server

```
Server running on port 5000
```

6) In another terminal, run the client a few time

```
1
jpo2022@arc
2
jpo2022@arc
3
jpo2022@arc
```

7) Take a screenshot that shows the results in the two terminals side by side and save it in the directory.

8) Create a bash script file called clients.sh that will prompt for the number or runs.
Note: Script files are just text files, but you have to set it to executable before running it.

```
#!/bin/bash

echo -n "Enter number of clients to run: "
read num

for ((i=1; i<=num; i++))
do
    java CounterClient &
done

wait

echo "All clients finished."
```

9) Run the script with 5 runs. You results may vary from mine, with values out of order, or even duplicates. Take a screenshot and save it in the directory.

```
jpo2022@archalienware ~/sandbox/taskw09 $ chmod +x clients.sh
jpo2022@archalienware ~/sandbox/taskw09 $ ./clients.sh
Enter number of clients to run: 5
4
5
6
7
8
All clients finished.
jpo2022@archalienware ~/sandbox/taskw09 $
```

10) Now, run the script with larger values, like 50 or 100, and see if you are seeing any duplicates. It may be hard to spot them if the exist.

11) Instead of sending it to the console, lets write the script to allow command line input and redirect the results to a file. Then you can run it as: `./client.sh 5`

```
#!/bin/bash

num=$1

if [ -z "$num" ]; then
    echo "Usage: ./clients.sh <number_of_clients>"
    exit 1
fi

for ((i=1; i<=num; i++))
do
    java CounterClient &
done | sort -n > results.txt

wait

echo "Results saved to results.txt"
```

12) Restart the server by using ctrl-c to terminate it, then run CounterServer again.

13) Run the client script with 100 or more, and check the results file to make sure you are getting results.

```
Results saved to results.txt
jpo2022@archalienware ~/sandbox/taskw09 $ ./clients.sh 500
Results saved to results.txt
jpo2022@archalienware ~/sandbox/taskw09 $
```

14) Use the Linux uniq command to find duplicates. If you are not getting duplicates, try again or with more executions. It's fairly likely you will get at least one duplicate at some point, but they may be rare. At 1000 executions, I had three duplicates. Take a screenshot of your duplicate values.

Part B – Network Traffic

While we are here, let's look at the network traffic that the server and client generates. It will help us better understand what happens during communication.

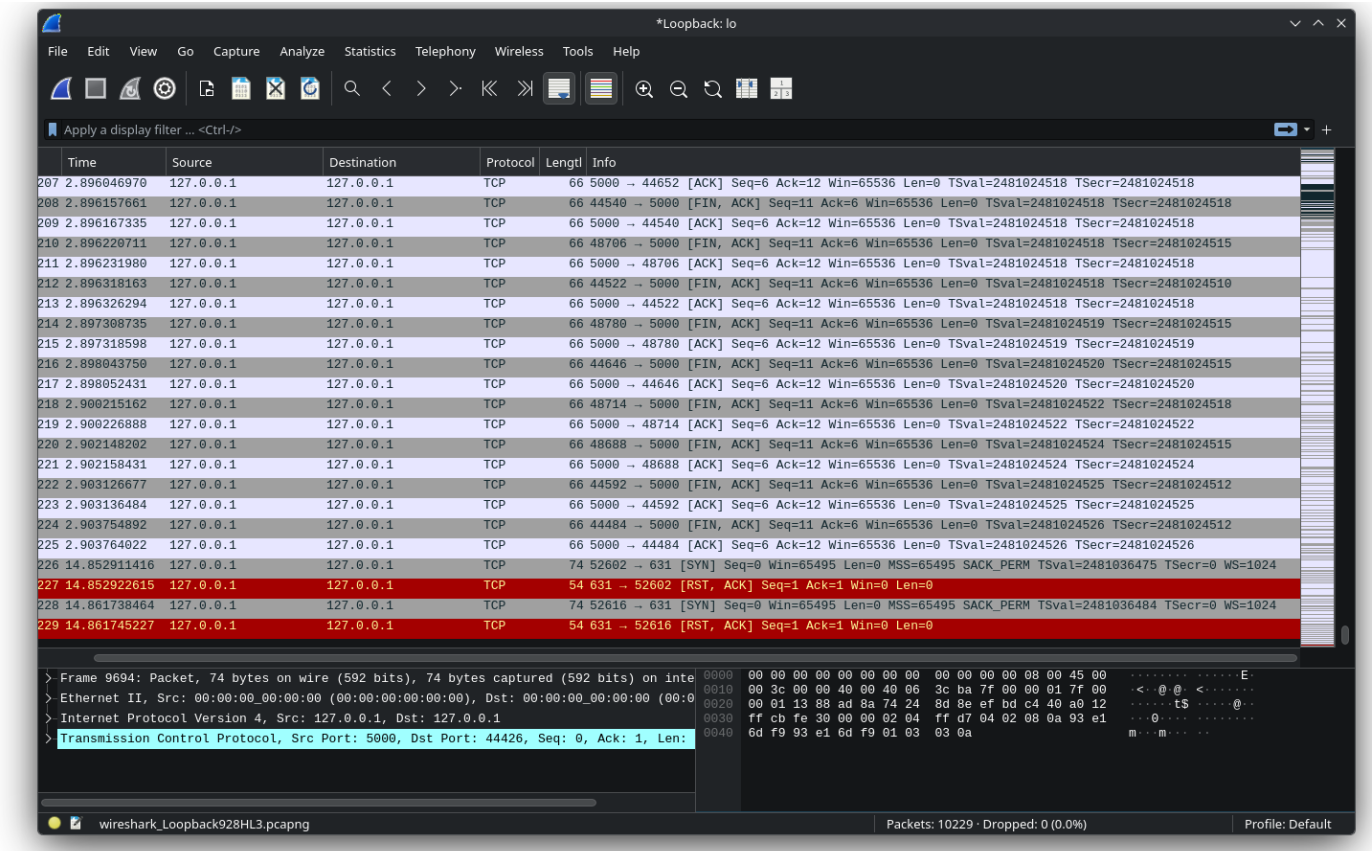
1) Start Wireshark

2) Select to capture Loopback: lo

3) Start capturing packets

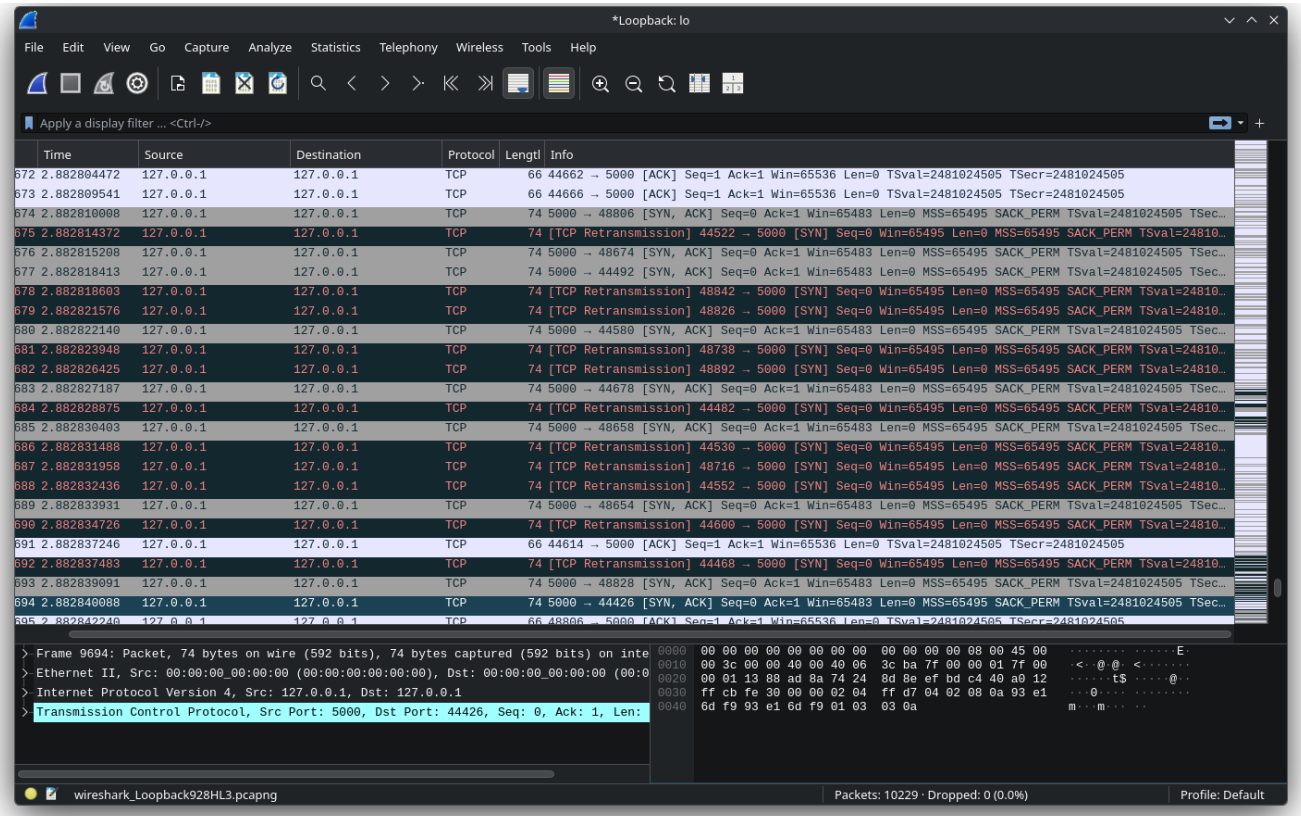
4) Start the CounterServer

5) Run the client scripting



6) When the client script is done, stop collecting packets, and save the file. Yours should similar to above.

7) Scan through the resulting packets, think about it, and study it. Do you see any retransmissions like mine below? Could it be these retransmissions causing the problem?



Part C

Of course, the resulting duplicate values from part A are an issue. For Part C you will need to make a copy of your Java code (do not delete the original files), and work on a solution to prevent the issue. In other words, when we start the server, then run the client script with x amount of executions), the results.txt file is guaranteed to have values 1 to x with no duplicates or missing values.

Once you have it solved, take a screenshot (or multiple) that demonstrate the working solution and save them in the directory.

Submission: Make sure to submit the taskw09 directory with all the code and screenshots in your 3970 work repository.